Answer the following questions

1)

What value is returned by the following function? Express your answer as a polynomial in n.

```
def NotSoFunny(n):
    r = 0
    for i = 1 to n
        for j = 1 to i
            for k = j to i + j
                r = r + 1
    return r
```

2)

Suppose I wish to augment the stack ADT with an operation `findMin`, that returns the smallest element in the stack in $\mathcal{O}(1)$ time. Thought for 9s...Here you go:

> *Maintain a pointer to the* ***min*** *element in the stack and update it as new elements are pushed.*

Does this idea work? If so, help me argue for the correctness. If not, indicate an edge case that breaks the idea. Also propose an alternate *fool-proof strategy.*

3)

A text editor's *Recent Documents* is implemented as a singly linked list, where the first node represents the most recently opened document. Every time a user opens a document already present in the list, that document's node must be moved to the front. The procedure below implements this "move to front" heuristic on a singly linked list with header node, where the node immediately after the header represents the most recently opened document. The code searches for a node whose `data` field equals `docID` and, if found, moves it to the front. If not found, the list is returned unchanged. Fill in the three missing lines.

```
SLL-MOVE-TO-FRONT(head, docID)
 1  p = head -> next
 2  q = head
 3  while (p != NULL && p -> data != docID)
 4        q = p
 5        p = p -> next
 6  if (p == NULL)
 7        return head
 8
 9
10
11  return head
```

4)

It is the last date of assignment submission and students are busy exchanging sheets beneath desks and covertly copying portions of each other's work. The teacher in the class wishes to identify the "*most social guy*" – the student who has copied from the most number of others. Whenever the teacher detects that student i has copied from student j's sheet, a method `copy_detected`(i, j) is called. Help your teacher design a data structure to:

- Record each detected copying instance efficiently via `copy_detected`(i, j).
- Track how many distinct students each student has copied from.
- Identify the "*high-risk plagiarizer*" (the student who has copied from the most number of others).

5)

Six pirates must divide a treasure loot of 300 gold coins among themselves as follows:

> *The senior pirate proposes a division. All pirates vote. If the senior pirate gets at least half of the votes, the division is final. Else, the senior pirate is killed and the next senior proposes.*

All pirates are rational: staying alive is the top priority; getting money is next; if indifferent, they vote to kill the proposer. How do you attack this problem from an algorithmic perspective? Design a data structure to:

i. Track which pirates are alive at each stage.
ii. Efficiently eliminate a pirate when voted out.
iii. Determine the next senior pirate after elimination.
iv. What is the final division? (Show each pirate's share.)